

Exame Modelo 2 — Paradigmas de Programação — Época Normal / Recurso 2025/2026

Instituição	P.PORTO — Escola Superior de Tecnologia e Gestão
Tipo de Prova	Exame Escrito — Época Normal / Recurso
Curso	Licenciatura em Engenharia Informática / Licenciatura em Segurança Informática em Redes de Computadores
Unidade Curricular	Paradigmas de Programação
Ano Letivo	2025/2026
Duração	2 horas

Observações

- Não é permitida a consulta.
- Não são permitidas questões relativas à Parte 2. Sempre que considerarem necessário, os alunos devem assumir os pressupostos que entenderem adequados, indicando-os explicitamente na resolução.
- É estritamente proibida a utilização de coleções da biblioteca `java.util`.

Parte 1 (6 Valores)

Pergunta 1 (1,5 valores)

Explique detalhadamente o conceito de encapsulamento em Programação Orientada a Objetos. Descreva como os modificadores de acesso (`private`, `protected`, `public`) contribuem para este princípio. Justifique por que razão o acesso direto aos atributos de uma classe é considerado uma má prática e apresente um exemplo prático que demonstre o uso correto de getters e setters com validação.

Pergunta 2 (1,5 valores)

Explique o conceito de polimorfismo em Java, distinguindo entre polimorfismo de sobrecarga (*overloading*) e polimorfismo de sobreposição (*overriding*). Descreva as regras que devem ser respeitadas em cada caso e forneça exemplos de código que ilustrem ambos os tipos de polimorfismo. Explique também como a JVM determina qual o método a invocar em tempo de execução no caso da sobreposição.

Pergunta 3 (1,5 valores)

Explique detalhadamente o conceito de exceções em Java. Distinga entre exceções verificadas (*checked exceptions*) e exceções não verificadas (*unchecked exceptions*). Descreva o funcionamento do mecanismo `try-catch-finally` e explique quando é adequado criar exceções personalizadas. Ilustre com um exemplo prático que demonstre a criação e o lançamento de uma exceção personalizada.

Pergunta 4 (1,5 valores)

Explique o significado e o comportamento do modificador `static` quando aplicado a variáveis e métodos. Discuta adicionalmente o impacto e a finalidade da utilização da palavra reservada `final` quando aplicada a: (1) uma classe, (2) um método e (3) uma variável. Distinga o comportamento de uma variável `final` quando armazena um tipo primitivo versus quando armazena uma referência de objeto.

Parte 2 (14 Valores)

1. Considere as seguintes interfaces `AidBox` e `Container`. A interface `AidBox` descreve as operações possíveis que definem o contrato de uma caixa de suprimentos utilizada na recolha de bens de uma instituição de ajuda humanitária. A interface `Container` define o contrato de um contentor associado a uma `AidBox`.

```
public interface AidBox {
    String getCode();
    String getZone();
    Container[] getContainers();
    boolean equals(Object obj);
}
```

```
public interface Container {
    String getCode();
    ItemType getType();
    double getCapacity();
    Measurement getLastMeasurement();
}
```

```
public enum ItemType {
    PERISHABLE_FOOD,
    NON_PERISHABLE_FOOD,
    CLOTHING,
    MEDICINE
}
```

```
public interface Measurement {
    double getValue();
}
```

```
}
```

Pergunta 1a (3 valores)

Considere a interface `AidBox` que representa uma caixa de suprimentos. Implemente a interface numa classe denominada `AidBoxImpl`. A `AidBox` deve possuir um array de `Container` com capacidade máxima de 4 contentores. Para a implementação do método `equals`, considere que **duas instâncias** de `AidBox` são iguais se possuírem o mesmo código (devolvido através do método `getCode()`). Implemente também um método `addContainer(Container container)` que adicione um contentor à `AidBox`, lançando uma `AidBoxException` caso a capacidade máxima seja atingida.

Pergunta 1b (2 valores)

Desenvolva o código necessário para testar a classe implementada (por exemplo, no contexto de um método `main`). Apresente um exemplo de teste para cada método implementado.

2. Considere a existência de uma classe `ReportImpl` que implemente a interface `Report`.

```
public interface Report {  
    String generate(IIInstitution inst);  
}
```

Pergunta 2a (4 valores)

Implemente os seguintes métodos na classe `ReportImpl` que podem ser utilizados para a geração do relatório:

```
int countContainersByType(AidBox aidbox, ItemType type);
```

- Este método deve devolver o número de contentores existentes na `AidBox` cujo tipo seja igual ao tipo recebido como argumento.

```
double getAverageOccupancy(AidBox aidbox);
```

- Este método deve devolver a média de ocupação de todos os contentores da `AidBox`. A ocupação de cada contentor é calculada através da fórmula: $(\text{últimaMedição} / \text{capacidade}) * 100$. Se um contentor não possuir medições, deve ser ignorado no cálculo da média.

Pergunta 2b (5 valores)

Na classe `ReportImpl`, implemente o método `generate`, gerando um relatório textual formatado.

Regras a considerar:

- O relatório deve percorrer todas as AidBoxes devolvidas pelo método `getAidBoxes()` da interface `IInstitution`.
 - Para cada AidBox, deve ser incluída a informação do código, zona e o número de contentores por cada tipo (utilizando o método `countContainersByType`).
 - Apenas devem ser incluídas AidBoxes cuja ocupação média (utilizando o método `getAverageOccupancy`) seja superior a 50%.
 - O método `generate` deve devolver uma String com todo o relatório formatado.
-

Excertos de Código Fornecidos

```
public class InstitutionImpl implements IInstitution {
    private AidBox[] aidBoxes;
    private int numberOfAidBoxes;
    private Vehicle[] vehicles;
    private int numberOfVehicles;

    public AidBox[] getAidBoxes() { return this.aidBoxes; }
    public Vehicle[] getVehicles() { return this.vehicles; }
}
```